DBMS Lab 2025-26 (Sem04)

Project Report

Problem Statement

1. Project Title

Hostel Management System

2. Background and Context

Educational institutions that provide residential facilities face significant challenges in managing hostel operations manually. The absence of an automated system leads to inefficiencies in student registration, fee collection, meal planning, inventory management, and grievance redressal. This project proposes a database-driven solution to address these challenges.

3. Problem Description

The existing hostel administration relies on manual records and disconnected spreadsheets, resulting in the following key problems:

3.1 Registration and Payment

- Student hostel seat allotment is done manually, leading to errors and duplication.
- There is no centralised system to track room assignments, check-in/check-out dates, or occupancy status.
- Fee payment records are maintained in physical registers, making retrieval and reconciliation time-consuming and error-prone.
- Generating dues reports and payment receipts requires significant manual effort.

3.2 Food Options, Bills, and Payment

- Students' individual meal preferences and dietary requirements are not systematically captured or tracked.
- Mess bills are computed manually based on attendance and chosen food plan, increasing the likelihood of billing mistakes.
- There is no integrated module to record and reconcile mess fee payments against outstanding balances.

3.3 Menu to Ingredient Forecast

- Daily and weekly menus are planned ad hoc without any systematic link to required ingredients or quantities.
- The absence of a menu-to-ingredient mapping makes it impossible to forecast grocery requirements accurately, leading to either wastage or shortages.

- Kitchen staff have no automated tool to scale ingredient quantities based on the actual number of students opting for a particular meal.

3.4 Forecast and Purchase of Groceries

- Grocery procurement is reactive rather than proactive, resulting in last-minute purchases and increased costs.
- There is no system to track current stock levels, minimum reorder points, or vendor details for perishable and non-perishable items.
- Purchase orders and vendor invoices are not linked to consumption data, preventing effective cost analysis.

3.5 Complaints Handling of Hostel and Mess

- Student complaints regarding hostel facilities (maintenance, cleanliness, utilities) and mess quality are registered informally with no tracking mechanism.
- There is no way to assign complaints to responsible staff, set resolution timelines, or escalate unresolved issues.
- Management lacks visibility into recurring complaint categories, preventing systemic improvements.

4. Proposed Solution

The proposed Hostel Management System is a PostgreSQL-based relational database application designed to automate and integrate all hostel operations. The system will be menu-driven, running on psql, and will utilise the following database concepts:

- Constraints, Indices, and Sequences for data integrity and performance.
- Aggregates and Nested Queries for reporting and analytics.
- Stored Procedures, Functions, Cursors, Triggers, and Control Loops for business logic automation.
- Exception Handling to ensure robust and fault-tolerant operations.

5. Objectives

- Automate student registration, room allocation, and fee management.
- Enable efficient food preference tracking, mess billing, and payment processing.
- Provide a menu-to-ingredient mapping to facilitate accurate grocery forecasting.
- Automate grocery procurement workflows, including stock monitoring and purchase order generation.
- Implement a structured complaint management module with assignment, tracking, and escalation capabilities.

6. Expected Outcome

On completion, the system will provide a fully integrated, menu-driven hostel management platform in PostgreSQL/psql that reduces manual workload, minimises errors, improves financial transparency, optimises food supply chain management, and ensures timely resolution of student grievances. The project will demonstrate mastery of core DBMS concepts as outlined in the Sem04 syllabus.

Hostel Management System

Relational Schema

DBMS Lab 2025-26 (Sem-04): PostgreSQL Project

Legend

PK	Primary Key
FK	Foreign Key
NN	NOT NULL
UQ	UNIQUE

Table Definitions

STUDENT

Core entity — one record per enrolled hostel student

Column	Type	Key	Notes / Constraints
student_id	SERIAL	PK	Auto-incremented primary key
name	VARCHAR(100)	NN	
email	VARCHAR(100)	UQ NN	
phone	CHAR(10)	NN	
gender	CHAR(1)		CHECK IN ('M','F','O')
dob	DATE		
address	TEXT		
admission_date	DATE		DEFAULT CURRENT_DATE

ROOM

Hostel rooms with capacity, occupancy and rent

Column	Type	Key	Notes / Constraints
room_id	SERIAL	PK	Auto-incremented primary key
room_no	VARCHAR(10)	UQ NN	
type	VARCHAR(20)		CHECK IN ('single','double','triple')
capacity	INT	NN	CHECK (capacity > 0)

current_occupancy	INT		DEFAULT 0
status	VARCHAR(20)		CHECK IN ('available','full','maintenance')
rent_per_month	NUMERIC(8,2)	NN	CHECK (rent_per_month > 0)

REGISTRATION

Links student to room — one active record per student at a time

Column	Type	Key	Notes / Constraints
reg_id	SERIAL	PK	Auto-incremented primary key
student_id	INT	FK	References STUDENT(student_id)
room_id	INT	FK	References ROOM(room_id)
check_in	DATE	NN	DEFAULT CURRENT_DATE
check_out	DATE		NULL until student leaves
status	VARCHAR(20)		CHECK IN ('active','checked_out')

PAYMENT

Room rent payment records linked to a registration

Column	Type	Key	Notes / Constraints
payment_id	SERIAL	PK	Auto-incremented primary key
reg_id	INT	FK	References REGISTRATION(reg_id)
amount	NUMERIC(8,2)	NN	CHECK (amount > 0)
payment_date	DATE		DEFAULT CURRENT_DATE
mode	VARCHAR(20)		CHECK IN ('cash','upi','card','online')
status	VARCHAR(20)		CHECK IN ('paid','pending','overdue')

MESS_SUBSCRIPTION

Student's mess enrollment and meal plan details

Column	Type	Key	Notes / Constraints
sub_id	SERIAL	PK	Auto-incremented primary key
student_id	INT	FK	References STUDENT(student_id)
meal_plan	VARCHAR(20)		CHECK IN ('full','veg','non-veg','partial')

start_date	DATE	NN	
end_date	DATE		NULL for ongoing subscription
monthly_fee	NUMERIC(8,2)	NN	CHECK (monthly_fee > 0)

MENU

Daily menu entries per meal type with expected student count

Column	Type	Key	Notes / Constraints
menu_id	SERIAL	PK	Auto-incremented primary key
menu_date	DATE	NN	
meal_type	VARCHAR(15)	NN	CHECK IN ('breakfast','lunch','dinner')
item_name	VARCHAR(100)	NN	
expected_students	INT		DEFAULT 0; drives ingredient forecast

INGREDIENT

Master ingredient list with quantity needed per student per serving

Column	Type	Key	Notes / Constraints
ingredient_id	SERIAL	PK	Auto-incremented primary key
name	VARCHAR(100)	UQ NN	
unit	VARCHAR(20)		e.g. kg, litre, pcs
qty_per_student	NUMERIC(6,3)	NN	CHECK (qty_per_student > 0)

MENU_INGREDIENT

Junction table — maps menu items to required ingredients with total quantity

Column	Type	Key	Notes / Constraints
menu_id	INT	FK PK	References MENU(menu_id); part of composite PK
ingredient_id	INT	FK PK	References INGREDIENT(ingredient_id); part of composite PK
total_qty	NUMERIC(8,3)		qty_per_student × expected_students

COMPLAINT

Student grievances for hostel, mess, or maintenance issues

Column	Type	Key	Notes / Constraints
complaint_id	SERIAL	PK	Auto-incremented primary key
student_id	INT	FK	References STUDENT(student_id)
category	VARCHAR(30)		CHECK IN ('hostel','mess','maintenance','other')
description	TEXT	NN	
filed_date	DATE		DEFAULT CURRENT_DATE
status	VARCHAR(20)		CHECK IN ('open','in_progress','resolved')
resolved_date	DATE		NULL until status = resolved

Hostel Management System

ER Diagram & Entity Reference

DBMS Lab 2025-26 (Sem04) · PostgreSQL Project

Legend

▶ `attribute_name (underlined)`	Primary Key (PK) — unique row identifier
`FK`	Foreign Key — references another entity's PK
`NN`	NOT NULL — value is mandatory
`UQ`	UNIQUE — no duplicate values allowed
`1:N`	One-to-many relationship

Entities & Attributes

Each entity below shows its attributes, data type, and key constraints. Primary keys are marked with ▸ and underlined.

STUDENT

Attribute	Data type
▸ student_id	SERIAL
name	VARCHAR(100)
email	VARCHAR(100)
phone	CHAR(10)
gender	CHAR(1)
dob	DATE
address	TEXT
admission_date	DATE

ROOM

Attribute	Data type	
▸ room_id	SERIAL	
room_no	VARCHAR(10)	
type	VARCHAR(20)	
capacity	INT	
current_occupancy	INT	
status	VARCHAR(20)	
rent_per_month	NUMERIC(8,2)	NN

REGISTRATION

Attribute	Data type
▸ reg_id	SERIAL
student_id	INT
room_id	INT
check_in	DATE
check_out	DATE
status	VARCHAR(20)

PAYMENT

Attribute	Data type
▸ payment_id	SERIAL
reg_id	INT
amount	NUMERIC(8,2)
payment_date	DATE
mode	VARCHAR(20)
status	VARCHAR(20)

MESS_SUBSCRIPTION

Attribute	Data type	
▸ sub_id	SERIAL	
student_id	INT	
meal_plan	VARCHAR(20)	
start_date	DATE	
end_date	DATE	
monthly_fee	NUMERIC(8,2)	NN

MENU

Attribute	Data type
▸ menu_id	SERIAL
menu_date	DATE
meal_type	VARCHAR(15)
item_name	VARCHAR(100)
expected_students	INT

INGREDIENT

Attribute	Data type
▸ ingredient_id	SERIAL
name	VARCHAR(100)
unit	VARCHAR(20)

MENU_INGREDIENT

Attribute	Data type
▸ menu_id	INT
▸ ingredient_id	INT
total_qty	NUMERIC(8,3)

COMPLAINT

Attribute	Data type
▸ complaint_id	SERIAL
student_id	INT
category	VARCHAR(30)

qty per student	NUMERIC(6,3)	**NN**

description	TEXT
filed_date	DATE
status	VARCHAR(20)
resolved_date	DATE

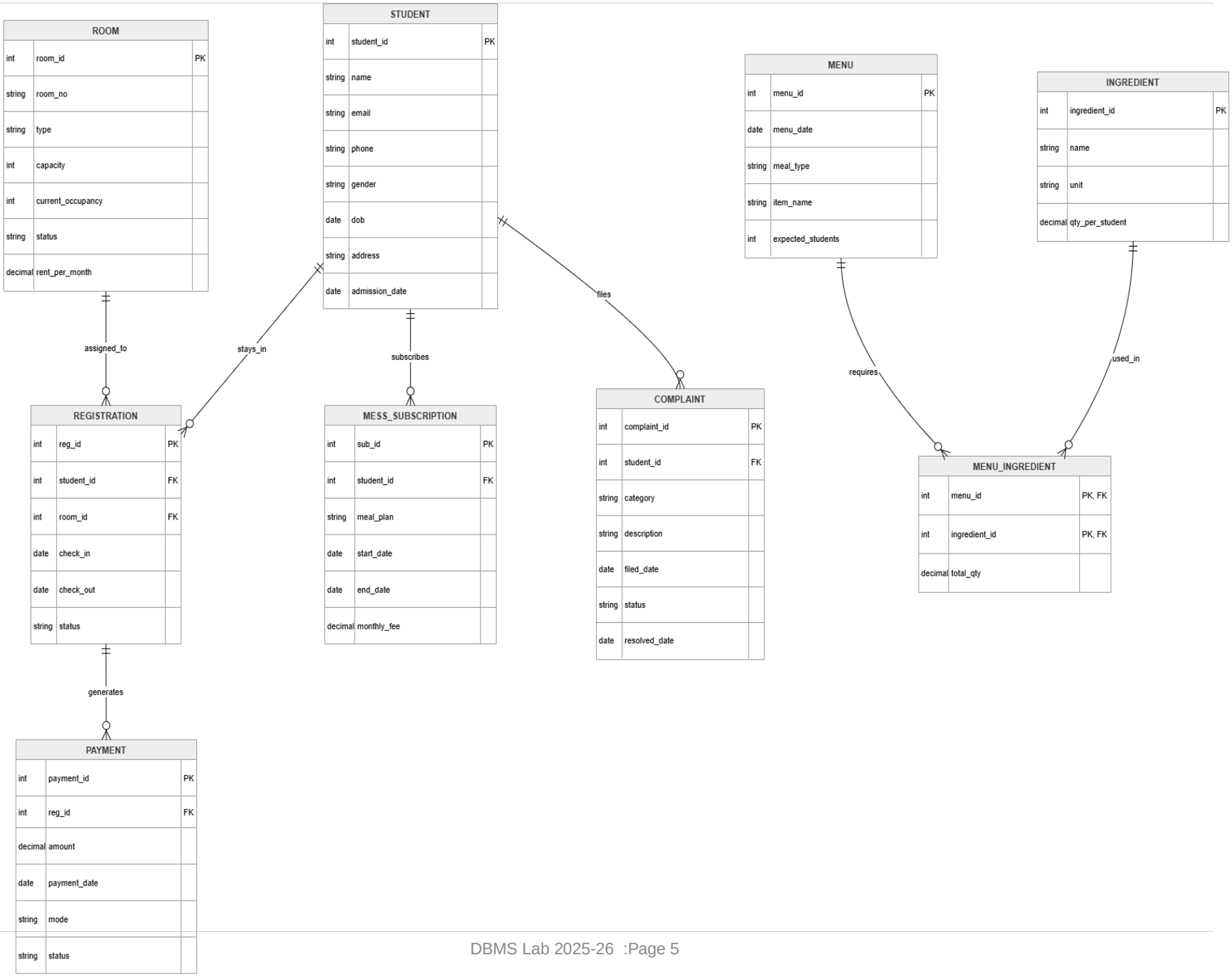
Relationships

All 7 relationships use one-to-many (1:N) cardinality with optional participation on the many side.

Entity 1	Cardinality	Entity 2	Relationship	Description
STUDENT	1 : N	REGISTRATION	stays in	One student can have multiple registrations over time
ROOM	1 : N	REGISTRATION	assigned to	One room can have multiple registrations (not simultaneously)
REGISTRATION	1 : N	PAYMENT	generates	One registration generates multiple monthly payments
STUDENT	1 : N	MESS_SUBSCRIPTION	subscribes to	One student can subscribe to mess in different periods
STUDENT	1 : N	COMPLAINT	files	One student can file multiple complaints
MENU	1 : N	MENU_INGREDIENT	requires	One menu item maps to multiple ingredients
INGREDIENT	1 : N	MENU_INGREDIENT	used in	One ingredient can appear in multiple menu items

Relationship summary

- STUDENT ——< REGISTRATION *[stays in]* One student can have multiple registrations over time
- ROOM ——< REGISTRATION *[assigned to]* One room can have multiple registrations (not simultaneously)
- REGISTRATION ——< PAYMENT *[generates]* One registration generates multiple monthly payments
- STUDENT ——< MESS_SUBSCRIPTION *[subscribes to]* One student can subscribe to mess in different periods
- STUDENT ——< COMPLAINT *[files]* One student can file multiple complaints
- MENU ——< MENU_INGREDIENT *[requires]* One menu item maps to multiple ingredients
- INGREDIENT ——< MENU_INGREDIENT *[used in]* One ingredient can appear in multiple menu items


ROOM
int room_id PK
string room_no
string type
int capacity
int current_occupancy
string status
decimal rent_per_month
STUDENT
int student_id PK
string name
string email
string phone
string gender
date dob
string address
date admission_date
MENU
int menu_id PK
date menu_date
string meal_type
string item_name
int expected_students
INGREDIENT
int ingredient_id PK
string name
string unit
decimal qty_per_student
assigned_to
stays_in
subscribes
files
requires
used_in
REGISTRATION
int reg_id PK
int student_id FK
int room_id FK
date check_in
date check_out
string status
MESS_SUBSCRIPTION
int sub_id PK
int student_id FK
string meal_plan
date start_date
date end_date
decimal monthly_fee
COMPLAINT
int complaint_id PK
int student_id FK
string category
string description
date filed_date
string status
date resolved_date
MENU_INGREDIENT
int menu_id PK, FK
int ingredient_id PK, FK
decimal total_qty
generates
PAYMENT
int payment_id PK
int reg_id FK
decimal amount
date payment_date
string mode
string status